PEP 691 – JSON-based Simple API for Python Package Indexes

Important

This PEP is a historical document. The up-to-date, canonical spec, Simple repository API, is maintained on the PyPA specs page.

See the PyPA specification update process for how to propose changes.

Abstract

The “Simple Repository API” that was defined in PEP 503 (and was in use much longer than that) has served us reasonably well for a very long time. However, the reliance on using HTML as the data exchange mechanism has several shortcomings.

There are two major issues with an HTML-based API:

- This means that to actually accept everything that is technically valid, tools have to pull in large dependencies or they have to rely on the standard library’s html.parser library, which is lighter weight but potentially doesn’t fully support HTML5.
- The primary issue with using a markup format designed for human consumption is that there’s not a great way to actually encode data within HTML. We’ve gotten around this by limiting the data we put in this API and being creative with how we can cram data into the API (for instance, hashes are embedded as URL fragments, adding the data-yanked attribute in PEP 592).

PEP 503 was largely an attempt to standardize what was already in use, so it did not propose any large changes to the API.

In the intervening years, we’ve regularly talked about an “API V2” that would re-envision the entire API of PyPI. However, due to limited time constraints, that effort has not gained much, if any, traction beyond people thinking that it would be nice to do.

This PEP attempts to take a different route. It doesn’t fundamentally change the overall API structure, but instead specifies a new serialization of the existing data contained in existing PEP 503 responses in a format that is easier for software to parse rather than using a human centric document format.

Goals

- Enable zero configuration discovery. Clients of the simple API MUST be able to gracefully determine whether a target repository supports this PEP without relying on any form of out of band communication (configuration, prior knowledge, etc). Individual clients MAY choose to require configuration to enable the use of this API, however.
- Enable clients to drop support for “legacy” HTML parsing. While it is expected that most clients will keep supporting HTML-only repositories for a while, if not forever, it should be possible for a client to choose to support only the new API formats and no longer invoke an HTML parser.
- Enable repositories to drop support for “legacy” HTML formats. Similar to clients, it is expected that most repositories will continue to support HTML responses for a long time, or forever. It should be possible for a repository to choose to only support the new formats.
- Maintain full support for existing HTML-only clients. We MUST not break existing clients that are accessing the API as a strictly PEP 503 API. The only exception to this, is if the repository itself has chosen to no longer support the HTML format.
- Minimal additional HTTP requests. Using this API MUST not drastically increase the amount of HTTP requests an installer must do in order to function. Ideally it will require 0 additional requests, but if needed it may require one or two additional requests (total, not per dependency).
- Minimal additional unique responses. Due to the nature of how large repositories like PyPI cache responses, this PEP should not introduce a significantly or combinatorially large number of additional unique responses that the repository may produce.
- Supports TUF. This PEP MUST be able to function within the bounds of what TUF can support (PEP 458), and must be able to be secured using it.
- Require only the standard library, or small external dependencies for clients. Parsing an API response should ideally require nothing but the standard library, however it would be acceptable to require a small, pure Python dependency.

Specification

To enable response parsing with only the standard library, this PEP specifies that all responses (besides the files themselves, and the HTML responses from PEP 503) should be serialized using JSON.

To enable zero configuration discovery and to minimize the amount of additional HTTP requests, this PEP extends PEP 503 such that all of the API endpoints (other than the files themselves) will utilize HTTP content negotiation to allow client and server to select the correct serialization format to serve, i.e. either HTML or JSON.

Versioning

Versioning will adhere to PEP 629 format (Major.Minor), which has defined the existing HTML responses to be 1.0. Since this PEP does not introduce new features into the API, rather it

describes a different serialization format for the existing features, this PEP does not change the existing 1.0 version, and instead just describes how to serialize that into JSON.

Similar to PEP 629, the major version number MUST be incremented if any changes to the new format would result in no longer being able to expect existing clients to meaningfully understand the format.

Likewise, the minor version MUST be incremented if features are added or removed from the format, but existing clients would be expected to continue to meaningfully understand the format.

Changes that would not result in existing clients being unable to meaningfully understand the format and which do not represent features being added or removed may occur without changing the version number.

This is intentionally vague, as this PEP believes it is best left up to future PEPs that make any changes to the API to investigate and decide whether or not that change should increment the major or minor version.

Future versions of the API may add things that can only be represented in a subset of the available serializations of that version. All serializations version numbers, within a major version, SHOULD be kept in sync, but the specifics of how a feature serializes into each format may differ, including whether or not that feature is present at all.

It is the intent of this PEP that the API should be thought of as URL endpoints that return data, whose interpretation is defined by the version of that data, and then serialized into the target serialization format.

JSON Serialization

The URL structure from PEP 503 still applies, as this PEP only adds an additional serialization format for the already existing API.

The following constraints apply to all JSON serialized responses described in this PEP:

- All JSON responses will always be a JSON object rather than an array or other type.
- While JSON doesn't natively support an URL type, any value that represents an URL in this API may be either absolute or relative as long as they point to the correct location. If relative, they are relative to the current URL as if it were HTML.
- Additional keys may be added to any dictionary objects in the API responses and clients MUST ignore keys that they don't understand.
- All JSON responses will have a meta key, which contains information related to the response itself, rather than the content of the response.
- All JSON responses will have a meta.api-version key, which will be a string that contains the PEP 629 Major.Minor version number, with the same fail/warn semantics as defined in PEP 629.

- All requirements of PEP 503 that are not HTML specific still apply.

Project List

The root URL / for this PEP (which represents the base URL) will be a JSON encoded dictionary which has a two keys:

- projects: An array where each entry is a dictionary with a single key, name, which represents string of the project name.
- meta: The general response metadata as described earlier.

As an example:

```
{
  "meta": {
    "api-version": "1.0"
  },
  "projects": [
    {"name": "Frob"},
    {"name": "spamspamspam"}
  ]
}
```

Note

The name field is the same as the one from PEP 503, which does not specify whether it is the non-normalized display name or the normalized name. In practice different implementations of these PEPs are choosing differently here, so relying on it being either non-normalized or normalized is relying on an implementation detail of the repository in question.

Note

While the projects key is an array, and thus is required to be in some kind of an order, neither PEP 503 nor this PEP requires any specific ordering nor that the ordering is consistent from one request to the next. Mentally this is best thought of as a set, but both JSON and HTML lack the functionality to have sets.

Project Detail

The format of this URL is /<project>/ where the <project> is replaced by the PEP 503 normalized name for that project, so a project named “Silly_Walk” would have a URL like /silly-walk/.

This URL must respond with a JSON encoded dictionary that has three keys: